ARHITECTURA PROIECT:

Raspberry Pi Pico 2

Role: Central microcontroller, manages audio processing, input/output control, and overall device logic.

Connections:
Display, Micro SD Card Module, DAC, Buttons, and Output components.

Display (ST7735R)

Interface: SPI

Connections:

- SDA (Data Line) connected to Raspberry Pi Pico 2 SPI pins
- SCL (Clock Line) connected to Raspberry Pi Pico 2 SPI pins

Role: Displays user interface, sequencing steps and device status.

Buttons

Interface: GPIO

Connections:

- Connected to GPIO pins for controls (playback, recording, navigation, sequencing)

Role: Allows user input for sampling, playback, sequence control, and settings.

Micro SD Card Module

Interface: SPI

Connections:

- SCK (Serial Clock) connected to Pico SPI pin
- MOSI (Master Out Slave In) connected to Pico SPI pin
- MISO (Master In Slave Out) connected to Pico SPI pin
- CS (Chip Select) connected to Pico GPIO pin

Role: Stores audio samples, sequences, presets, and configuration data.

DAC (MAX98357A)

Interface: I2S

Connections:

- Connected via I2S interface pins on the Raspberry Pi Pico 2

Role: Converts digital audio signals to high-quality analog audio for speaker output.

Output (Speaker)

Role: Outputs audio signal from the DAC to provide sound playback.

Hardware

- Raspberry Pi Pico 2 (RP2350): Main microcontroller handling audio processing and system logic.
- I2S DAC – MAX98357A: Converts digital audio signals to analog for high-quality speaker output.
- ST7735R display: Colored screen for animations.
- 3.5mm socket: Outputs audio playback. (Headphones not included!)
- Buttons: Input for interraction with the device.

SOFTWARE:

Library	Description	Usage
embassy	Asynchronous embedded framework for Rust	Core framework for running async tasks on the Raspberry Pi Pico 2
embassy-rp	Embassy HAL for RP2040	Hardware abstraction layer for the Raspberry Pi Pico 2

Library	Description	Usage
embedded-hal	Hardware Abstraction Layer (HAL) traits	Provides unified interfaces for hardware drivers
embedded-graphics	2D graphics library	Used for drawing to the ST7735R display
st7735-lcd	Rust driver for ST7735 display	Controls the ST7735R display
[embassy_rp::pio_programs::i2s]	PIO-backed I²S output driver	Enables I²S audio output using the Programmable I/O system

EXERCITII:

1. Diferențe de principiu – microprocesor vs microcontroller

Comparation

Characteristic	Microcontroller	Microprocessor
Function	Includes CPU, mem & I/O	Includes only the CPU
Cost	>> ***cheaper***	>> ***expensive***
Complexity	>> ***simple***	>> ***complex***
Use case	***Incorporated devices***	***PCs, Servers, Laptops***



Microcontroller (MCU)

Integrated in embedded systems for certain tasks

- low operating frequency (MHz)
- a lot of I/O ports
- controls hardware
- does not require an Operating System
- costs $0.1 - $25
- annual demand is billions

Microprocessor (CPU)

General purpose, for PC & workstations

- high operating frequency (GHz)
- limited number of I/O ports
- usually requires an Operating System
- costs $75 - $500
- annual demand is tens of millions

2. Presupunem că avem un semnal PWM generat de Timer 1 configurat în mod *Fast PWM 8-bit*, non-inverting, cu prescaler 64, cu OCR1A setat la 64, iar frecvența de ceas a uC este de 16 MHz.
   Calculați durata de timp pentru care semnalul este HIGH în cadrul unui ciclu (HIGH - LOW).

1. **Frecvența „tic-ului” timer-ului**

$$f_{\text{tick}} = \frac{f_{\text{CPU}}}{\text{prescaler}} = \frac{16\ \text{MHz}}{64} = 250\ \text{kHz}$$

2. **Perioada unui tic**

$$T_{\text{tick}} = \frac{1}{f_{\text{tick}}} = \frac{1}{250\ \text{kHz}} = 4\ \mu\text{s}$$

3. **Mod Fast-PWM 8-bit, non-inverting**

   Contorul numără de la 0 la 255 (256 poziții).

   În mod non-inverting, ieșirea este setată HIGH la **BOTTOM (0)** și este trecută LOW atunci când contorul atinge valoarea **OCR1A**.

4. **Număr de tic-uri cu nivel HIGH**

$$N_{\text{HIGH}} = \text{OCR1A} = 64$$

   (ieșirea este HIGH pentru contor 0…63; 64/256 ≈ 25 % duty-cycle)

5. **Durata în timp a nivelului HIGH**

$$t_{\text{HIGH}} = N_{\text{HIGH}} \times T_{\text{tick}} = 64 \times 4\ \mu\text{s} = 256\ \mu\text{s}$$

3. Avem doua CPU-uri cu următoarele caracteristici, având același set de instrucțiuni:
   • **CPU 1**: Frecvența ceasului = 2GHz, CPI (cicluri medii pe instrucțiune) = 1,5
   • **CPU 2**: Frecvența ceasului = 1600MHz, CPI = 1
   Care dintre cele două este mai performant?

2. Performanța relativă

$$\text{CPU 1: } \frac{f}{\text{CPI}} = \frac{2\ \text{GHz}}{1,5} \approx 1,33$$
$$\text{CPU 2: } \frac{f}{\text{CPI}} = \frac{1,6\ \text{GHz}}{1} = 1,60$$

3. Concluzie

$$\frac{\text{Performanță}_{\text{CPU 2}}}{\text{Performanță}_{\text{CPU 1}}} = \frac{1,60}{1,33} \approx 1,2$$

CPU 2 este cu ~20 % mai performant decât CPU 1, în condițiile aceloraşi instrucțiuni.

4. Aveti de implementat un sistem ce necesita achizitia de date la frecventa mica de la un numar mare de senzori digitali. Aveti la dispozitie un ATmega328P si libertatea de a alege tipul de senzori si interfata de comunicarea cu acestia.
   Ce interfata de comunicare alegeti?
   Care este numarul maxim de senzori pe care ii puteti conecta?

De ce I²C (TWI) este alegerea optimă

1. **Două fire** indiferent de numărul de dispozitive (SDA + SCL).
2. **Adresare pe 7 biți** – fiecare senzor primește un ID unic pe bus.
3. Viteza standard (100 kHz) este mai mult decât suficientă pentru achiziții lente.
4. Controlerul I²C este implementat hardware în ATmega328P, deci încărcarea CPU este minimă.

Numărul maxim de senzori

- Adrese posibile pe 7 biți: `0x00 … 0x7F` → 128 combinații.
- Adrese rezervate:
  - `0x00` (general call / START byte)
  - `0x01 … 0x07` (funcții speciale)
  - `0x78 … 0x7F` (prefix pentru adrese 10 biți)

$$128 - \left[(1) + (7) + (8)\right] = 112$$

→ **Se pot conecta până la 112 senzori digitali I²C cu adrese unice** pe același bus, fără multiplexoare suplimentare.
Dacă unii senzori împart aceeași adresă, se poate depăși acest număr folosind multiplexoare I²C (ex. TCA9548A), dar nu era cerut în problemă.

5. Numărul de accidente ce implică utilizatorii de trotinete este în creștere. Faceți parte dintr-o echipă de ingineri ce își propune să proiecteze un sistem ce utilizează un senzor de tip **IMU** (Inertial Measurement Unit) cu ajutorul căruia să detecteze evenimente de tip accident (impact/ cădătură) și să trimită mai departe comanda de declanșare la un **airbag** special proiectat pentru a proteja utilizatorii de trotinete.
   Aveți la dispoziție:
   – **ATmega328P**; **Senzor IMU** la alegere (minim 6 axe – 3 × accelerometru, 3 × giroscop); **componente pasive** la alegere; **alimentare** de la o sursă DC de 24 V; stabilizatoare de tensiune la alegere;
   – Un bloc hardware **„comandă airbag”** construit de colegii de echipă – care, pentru o comandă de declanșare, trebuie să primească un semnal de tipul „1” pentru **10 µs**.
6. **6.1 [3p]** Propuneți o schemă bloc.



6.2

Solutie hardware:
Circuit specializat (ex. **TPS3823**, **MAX706**) conectat la pinul RESET al ATmega328P. MCU trimite un „puls de viaţă” (“kick”) la fiecare 10 ms pe o ieşire GPIO. Dacă pulsul nu mai apare, supraveghetorul trasează RESET după **t_{WD-ext} ≈ 25 ms**.

Software:

Un contor global missed_ticks incrementat în main(); ISR-ul de 1 kHz îl resetează. Dacă depăşeşte 3, se intră automat în safe_mode() care declanşează airbag-ul manual şi resetează MCU.

6.3

Aveți de propus o soluție alternativă pentru blocul hardware **„comanda airbag”**, bazată tot pe 328P. Veți primi această comandă pe o **intrare digitală** – și aveți de comandat pe ieșire un **tranzistor**, tot prin intermediul unui GPIO.

Propuneți o **soluție software** pentru a minimiza timpul scurs între primirea comenzii digitale și trimiterea comenzii spre tranzistor – și **calculați care este valoarea minimă** pentru acest timp.

1. Strategia de minimizare a latenței

Măsură	Detaliu	Motiv
a. Interrupt extern pe front (INT0/INT1)	Configurăm pinul de intrare ca **INT0** (PD2) pe front ascendent.	Evenimentul este preluat de controlerul de întreruperi hardware, evitând polling-ul.
b. MCU în modul *IDLE*	Bucla `main()` intră în `sleep_cpu();`	În *IDLE* pipeline-ul este gol ⇒ vectorizarea întreruperii începe imediat, fără a aștepta terminarea unei instrucțiuni lungi.
c. ISR ultra-scurtă (doar SBI)	ISR conține un singur `SBI PORTD,7` (set bit) care deschide tranzistorul.	2 cicluri de ceas ⇒ ~125 ns la 16 MHz.
d. Acces direct la registru	Nu se folosește `digitalWrite()`, ci instrucțiuni pe port.	Elimină apelurile de funcție și verificările.

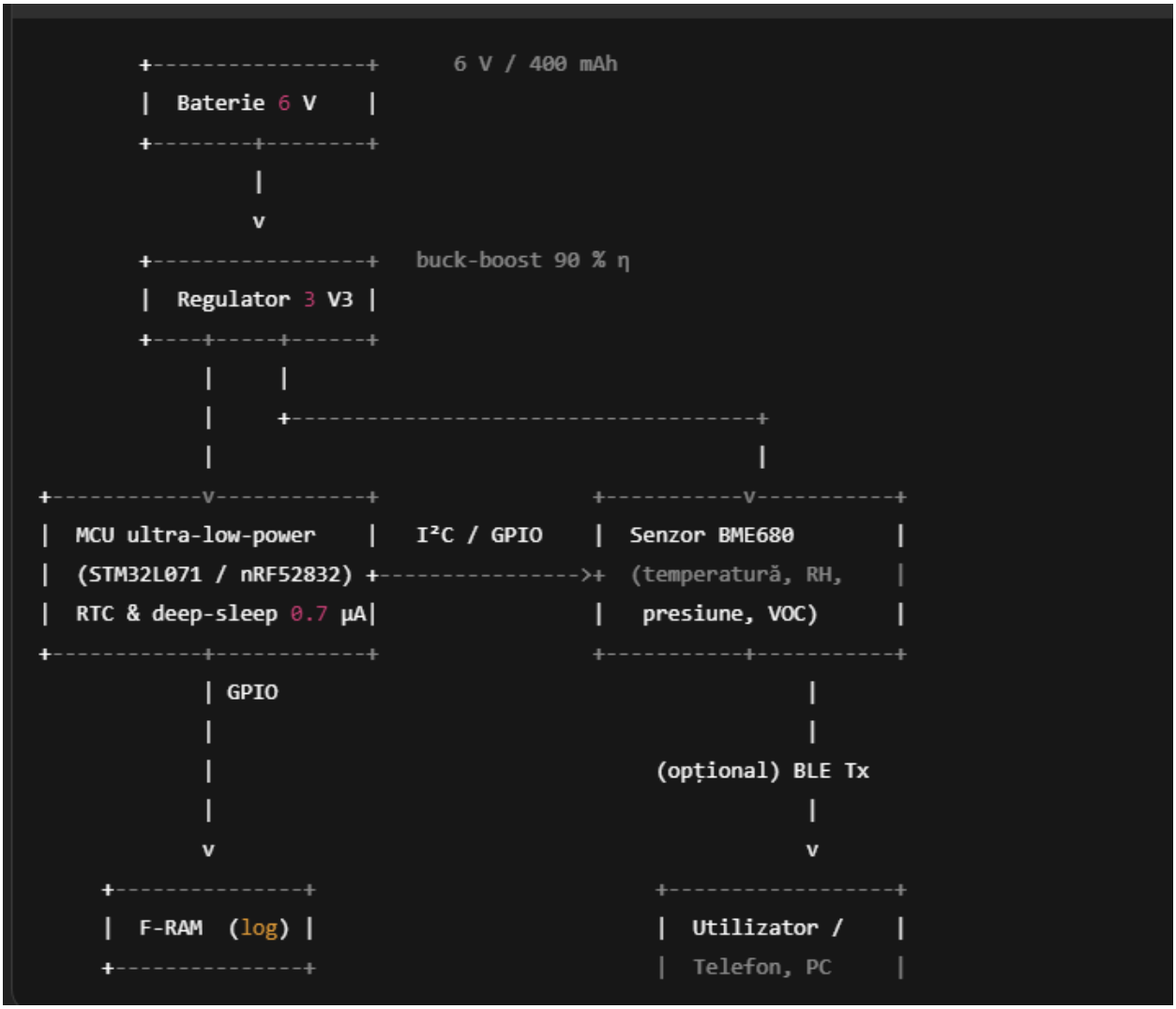
3. Calculul timpului minim

Segment	Cicluri	Timp (16 MHz)
Sincronizare & setare flag	≈ 2 cicluri	2 × 62,5 ns = **125 ns**
Vectorizare întrerupere (fetch + push PC + JMP)	5 cicluri	5 × 62,5 ns = **312 ns**
Prima instr. ISR (`SBI`)	2 cicluri	2 × 62,5 ns = **125 ns**
Total	**≈ 9 cicluri**	**≈ 562 ns**

7 . Vreți să monitorizați gradul de poluare de la birou. Decideți să implementați propriul senzor de poluare ce monitorizează o serie de parametri de mediu la alegerea voastră.
Nu doriți să fie alimentat de la priză, dar nici să schimbați bateria prea des – drept urmare va trebui să implementați un sistem hardware-software cât mai eficient.

Aveți la dispoziție o baterie de 6 V / 400 mAh.

Propuneți o implementare hardware & software (pe scurt, la nivel de scheme bloc) și **calculați durata maximă de funcționare** presupunând un interval de **60 s** între eșantionări.



3. Curent mediu și autonomie

- Curent mediu

$$I_{\text{avg}} = \frac{2{,}40\ \text{mAs}}{60\ \text{s}} = 0{,}040\ \text{mA} = 40\ \mu\text{A}$$

- Capacitate utilă după randament 90 %

$$Q_{\text{eff}} = 400\ \text{mAh} \times 0{,}90 = 360\ \text{mAh}$$

- Durata maximă de funcționare

$$T = \frac{Q_{\text{eff}}}{I_{\text{avg}}} = \frac{360\ \text{mAh}}{0{,}040\ \text{mA}} = 9000\ \text{h} \approx 375\ \text{zile} \approx 1{,}0\ \text{an}$$